The math you need this week

Vectors · distance · mean, min, max · standard deviation · about 15 minutes

Before you start

This is a **review**, not new material. Week 2 uses these ideas all the time. Read this file, then do the **Week 2 Math check** on Moodle (no grade, as many tries as you like). If a question goes wrong, the feedback tells you which section to read again.

Our examples use the Week 1 fish, with two features: *lightness* and *width*.

M2.1 A feature vector is a list of numbers

A **vector** is an ordered list of numbers. In this course, one object (one fish, one flower) becomes one vector: its **feature vector**. Each number is a **component**, and the number of components is d, the number of features.

Worked example

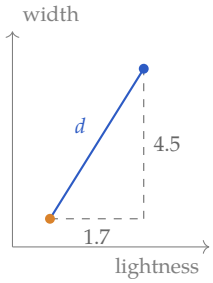
A salmon with lightness 2.5 and width 16.0 is the vector $\mathbf{x} = (2.5,\ 16.0)$. Its components are $x_1 = 2.5$ and $x_2 = 16.0$, and $d = 2$. The order matters: $(16.0,\ 2.5)$ would mean lightness 16.0 and width 2.5, a different fish.

In NumPy

```
import numpy as np
x = np.array([2.5, 16.0])   # x[0] is lightness, x[1] is width
```

M2.2 Distance between two vectors

How far apart are two fish in feature space? Draw them as points. The straight line between them is the hypotenuse of a right triangle, so Pythagoras gives its length. In d dimensions we do the same thing for every component: subtract, square, add, take the square root.



Euclidean distance

$$\|\mathbf{a} - \mathbf{b}\| = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2 + \cdots + (a_d - b_d)^2}$$

Worked example

Salmon $\mathbf{a} = (2.5,\ 16.0)$ and sea bass $\mathbf{b} = (4.2,\ 20.5)$.
Differences: $2.5 - 4.2 = -1.7$ and $16.0 - 20.5 = -4.5$. Squares: 2.89 and 20.25 (a square is never negative).
Sum: 23.14. Square root: $\sqrt{23.14} \approx \mathbf{4.81}$.

In NumPy

```
a = np.array([2.5, 16.0]);  b = np.array([4.2, 20.5])
np.sqrt(np.sum((a - b)**2))   # 4.81, step by step
np.linalg.norm(a - b)         # 4.81, the same in one call
```

M2.3 The mean, minimum, maximum and range

The **mean** (average) of n numbers is their sum divided by n. The symbol $\sum$ ("sigma") just means *add them all up*.

Mean

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{x_1 + x_2 + \cdots + x_n}{n}$$

Worked example

Lightness of the five salmon: 2.5, 3.0, 3.8, 4.5, 5.6. Sum $= 19.4$, and $n = 5$, so $\bar{x} = 19.4/5 = \mathbf{3.88}$. Two more simple numbers: the smallest value min $= 2.5$, the largest max $= 5.6$, and the **range** $=$ max $-$ min $= \mathbf{3.1}$.

In NumPy

```
s = np.array([2.5, 3.0, 3.8, 4.5, 5.6])
np.mean(s)                  # 3.88
s.min(), s.max(), np.ptp(s)  # 2.5, 5.6, 3.1  (ptp = range)
```

M2.4 The standard deviation

The **standard deviation** measures how far the numbers are, typically, from their mean. Small: the values are close together. Large: they are spread out.

Standard deviation

$$\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

In words: subtract the mean, square, take the mean of the squares, take the square root.

Worked example

The same five salmon, with $\bar{x} = 3.88$:

x_i	2.5	3.0	3.8	4.5	5.6
$x_i - \bar{x}$	-1.38	-0.88	-0.08	0.62	1.72
$(x_i - \bar{x})^2$	1.9044	0.7744	0.0064	0.3844	2.9584

Sum of squares $= 6.028$. Divide by $n = 5$: 1.2056 (this is the **variance** σ^2). Square root: $\sigma \approx \mathbf{1.10}$.

In NumPy

```
np.std(s)     # 1.098 -- divides by n, like our formula
```

Be careful

pandas' `.std()` divides by $n - 1$ instead of n, so on the same five numbers it gives 1.23. In this course we divide by n, as `np.std` and scikit-learn's `StandardScaler` do.

M2.5 Why this matters in Week 2

In Week 2 the computer decides by *distance*: a fish belongs to the class whose centre is closest. Distance adds the squared differences of all features, so a feature with large numbers can take over. Measure the width in millimetres instead of centimetres (16.0 becomes 160) and the distance between our two fish jumps from 4.81 to about **45.0**, almost all of it from width. Week 2 fixes this with *normalization*, built from the numbers on this page.